

Lecture 3

**Web Services, Replication, Cloud
Computing & Load Balancing**

COMP2211: Distributed Systems

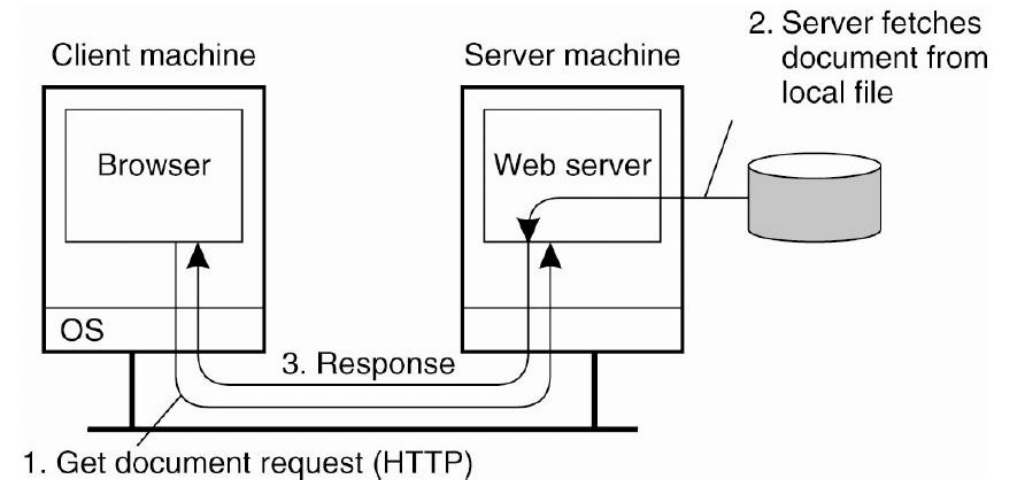
Thiru Vasantam

Topic 3: Web Services

Client-Server Interaction

A file is accessed through a Unique Resource Locator (URL):

- It specifies the DNS name of its associated server along with a file name
- Documents are exchanged using Hypertext Transport Protocol (HTTP)
- Browser takes input from users and displays received documents; it also follows guidelines on how to process and display the data (HTML, JavaScript, Images)

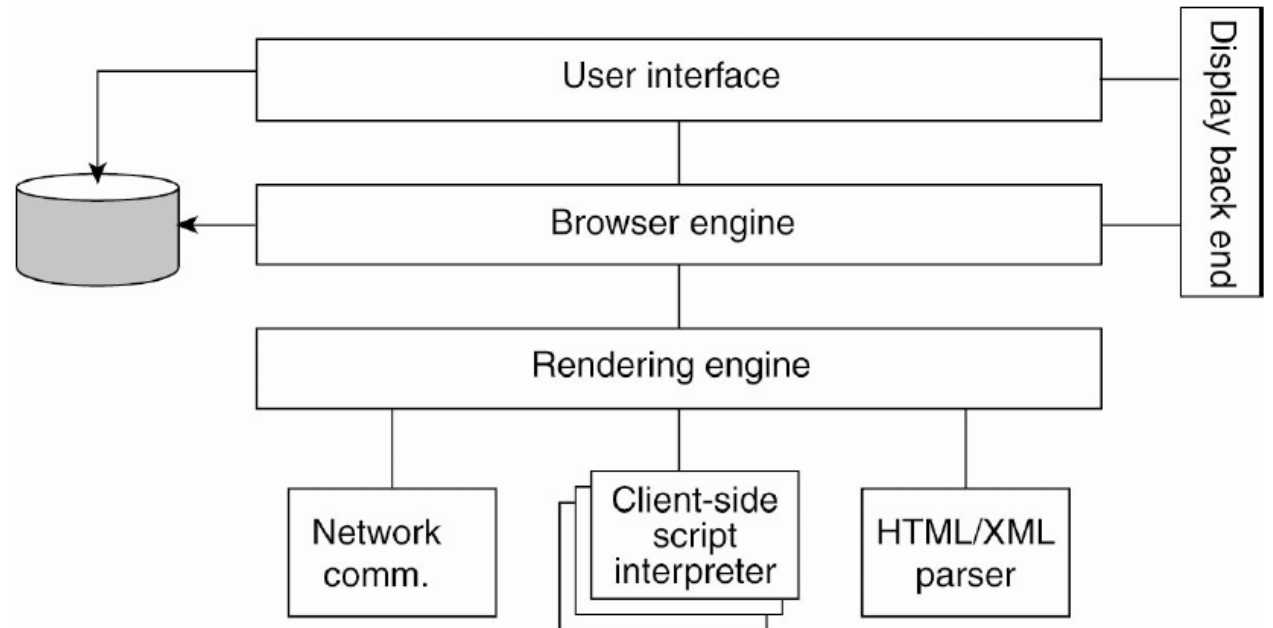


Source: Steen & Tannenbaum textbook

Web Browser Clients

The logical components of a web browser

- **User interface:** user submits a request and interacts with the visual elements of the web page
- **Browser engine:** forwards the request to the rendering engine
- **Rendering engine:** transforms the content (HTML, XML, Images) into visual representations for displaying on the screen for users
- **Client-sided script interpreter:** executes JavaScript code embedded in a website and then forwards it to the rendering engine
- **HTML/XML:** converts text into HTML/XML objects



Source: Steen & Tannenbaum textbook

Main Components of Web Services

- Use well-known web standards for distributed computing

Communication

- Message content expressed in XML
- Simple Object Access Protocol (SOAP)
 - Lightweight protocol for sync/async communication
 - Enables communication between client and server
 - Defines rules for using XML to represent request/reply messages and for communicating documents
 - SOAP message has: a reference to service description, message, documents,

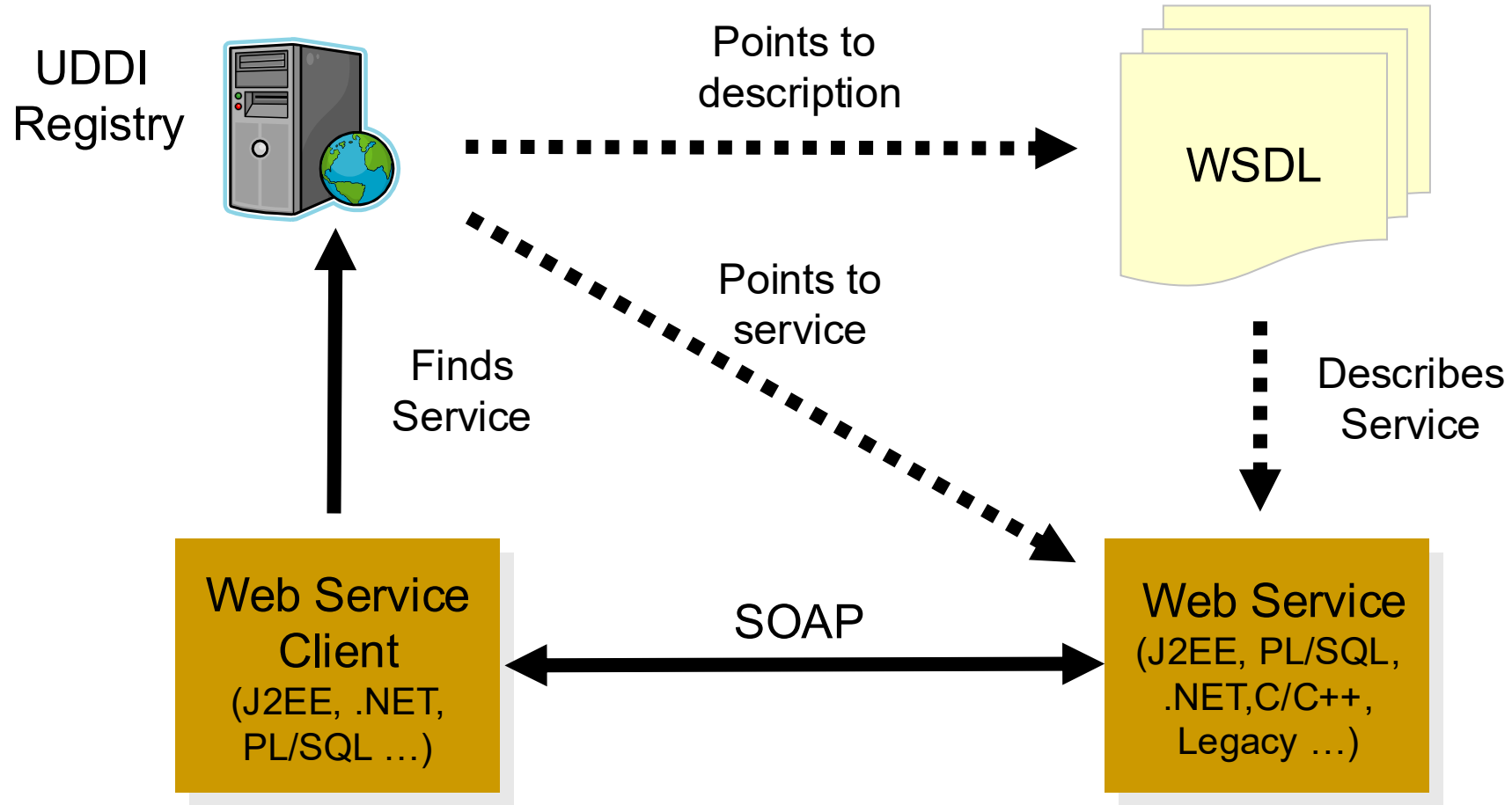
Service Description

- Web Services Description Language (WSDL)
 - Interface description for web services, and also has the details of the communication method and URL

Service Discovery

- Universal Description Discovery and Integration (UDDI): uses name, URL, and attributes to find web services
 - Directory with web service description in WSDL

Typical Web Services Architecture



Attributes of Web-Services

- **Web-based Protocols** : Web-services based on HTTP are designed to work over the public internet. The use of HTTP for transport means these protocols can **traverse firewalls**, and can work in a **heterogeneous environment**.
- **Interoperability** : SOAP defines a common standard that allows **differing systems to interoperate**. E.g., the tooling allows Web clients to access Java server components and vice versa.
- **XML-based** : The Extensible Markup Language (XML) is a standard framework for creating **machine-readable documents**.

Complicated structure

REST + JSON

- **REST** (**RE**presentational **S**tate **T**ransfer): replacement for SOAP
 - an architectural style, treating the Web as a resource-centric application
 - Each URL in a RESTful application represents a resource
 - We can use HTTP operations to manipulate data, ex: GET, DELETE, PUT, POST, etc.
 - Examples:
 - <http://www.bookstore.com/books/> -- a list of books
 - <http://www.bookstore.com/books/0321396855/> -- details about a specific book with the ISBN 0321396855
(as <http://www.bookstore.com/action/query?t=b&id=11117645532&qp=0321396855>)
- **JSON** (JavaScript Object Notation)
 - an open standard format that uses human-readable text to transmit data objects consisting of attribute–value pairs (alternative to XML)
 - provide lightweight communication (v/s. SOAP / XML)

Compare XML and JSON representations

```
<?xml version="1.0" encoding="UTF-8"?>
<authentication-context>
  <username>my_username</username>
  <password>my_password</password>
  <validation-factors>
    <validation-factor>
      <name>remote_address</name>
      <value>127.0.0.1</value>
    </validation-factor>
  </validation-factors>
</authentication-context>
```

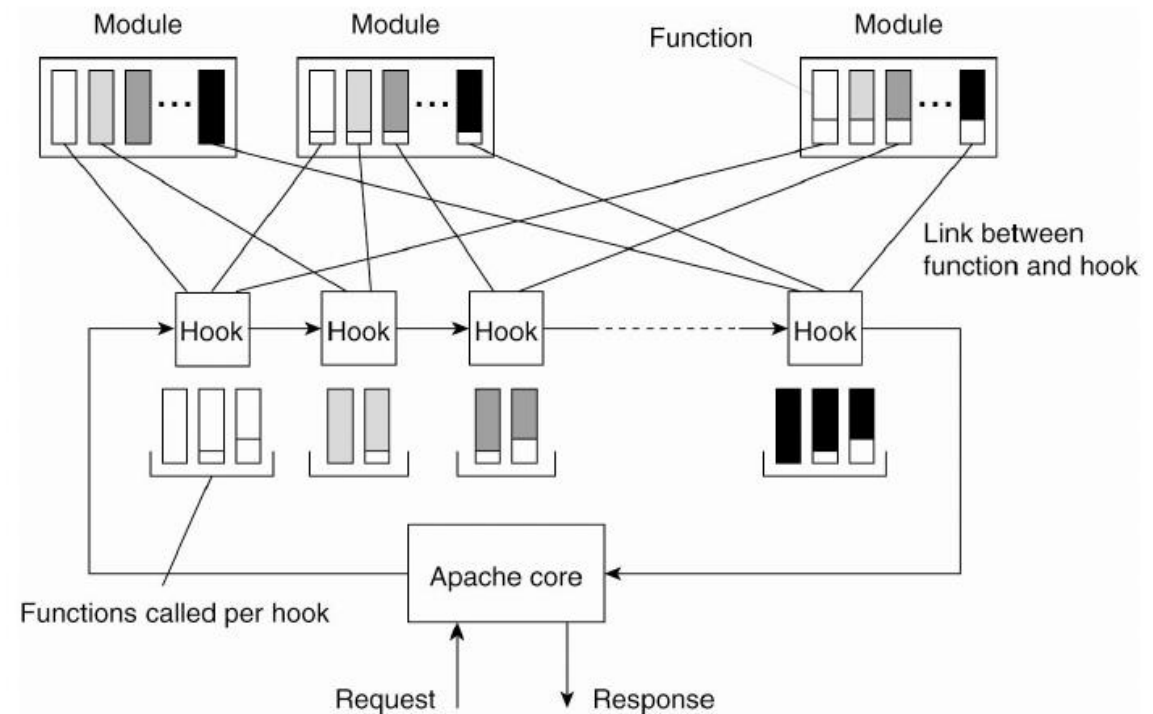
XML

```
{
  "username" : "my_username",
  "password" : "my_password",
  "validation-factors" : {
    "validationFactors" : [
      {
        "name" : "remote_address",
        "value" : "127.0.0.1"
      }
    ]
  }
}
```

JSON

The Apache Web Server: Architecture

- Apache web servers are popular
- Module has one or more functions
- Functions are invoked using hooks; every function has one hook
- For each request, a set of hooks are processed sequentially



Source: Steen & Tannenbaum textbook

Pyro4 with Web Service – Sample Server

```
import Pyro4
import requests

class Weather(object):
    def __init__(self):
        self.weather_data = None

    @Pyro4.expose
    def update_weather(self):
        url =
            "http://api.openweathermap.org/data/2.5/weather?q=city_name&appid=your_api_key"
        try:
            response = requests.get(url)
            response.raise_for_status()
            self.weather_data = response.json()
        except requests.exceptions.RequestException as e:
            self.weather_data = "Error: " + str(e)

    @Pyro4.expose
    def get_weather(self):
        return self.weather_data

def start_server():
    weather = Weather()
    daemon = Pyro4.Daemon()
    ns = Pyro4.locateNS()
    uri = daemon.register(weather)
    ns.register("example.weather", uri)
    print("Server started.")
    daemon.requestLoop()

if __name__ == '__main__':
    start_server()
```

Pyro4 with Web Service – Sample Client

```
import Pyro4

def start_client():
    ns = Pyro4.locateNS()
    uri = ns.lookup("example.weather")
    weather = Pyro4.Proxy(uri)

    weather.update_weather()
    print(weather.get_weather())

if __name__ == '__main__':
    start_client()
```

Note:

- The server script defines a **Weather** class that has two methods: **update_weather** and **get_weather**.
- The **update_weather** method uses the **requests** library to make a GET request to a public weather API to retrieve the current weather information for a specific city.
- The returned JSON data is then stored in the **weather_data** attribute of the **Weather** object. The **get_weather** method returns the **weather_data** attribute.

Proxy Servers

- Use a Web proxy when the browser does not speak File Transfer Protocol (FTP) (or for caching and offloading)
- Proxy servers can be used to store or cache data/files near users

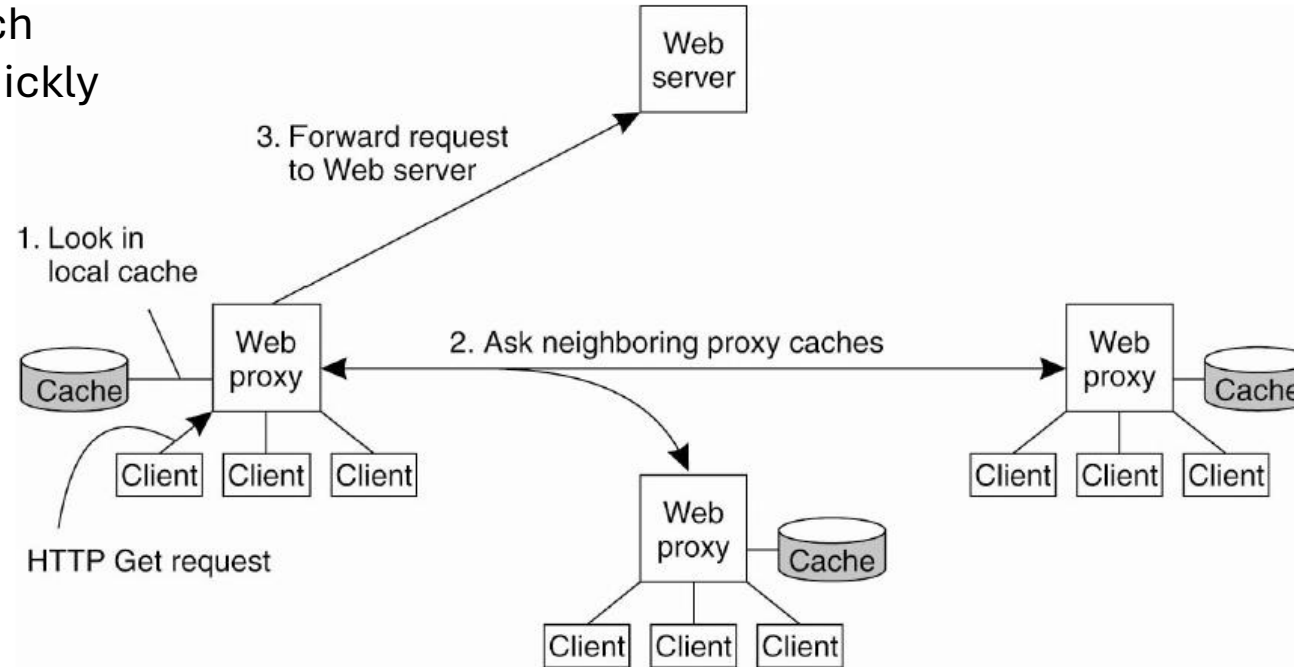


Source: Steen & Tannenbaum textbook

Cooperative Catching

The principle of cooperative catching

- Proxies cooperate with each other to serve a request quickly



Source: Steen & Tannenbaum textbook

Challenges

Question: Can you think of any challenges in using Web proxies?

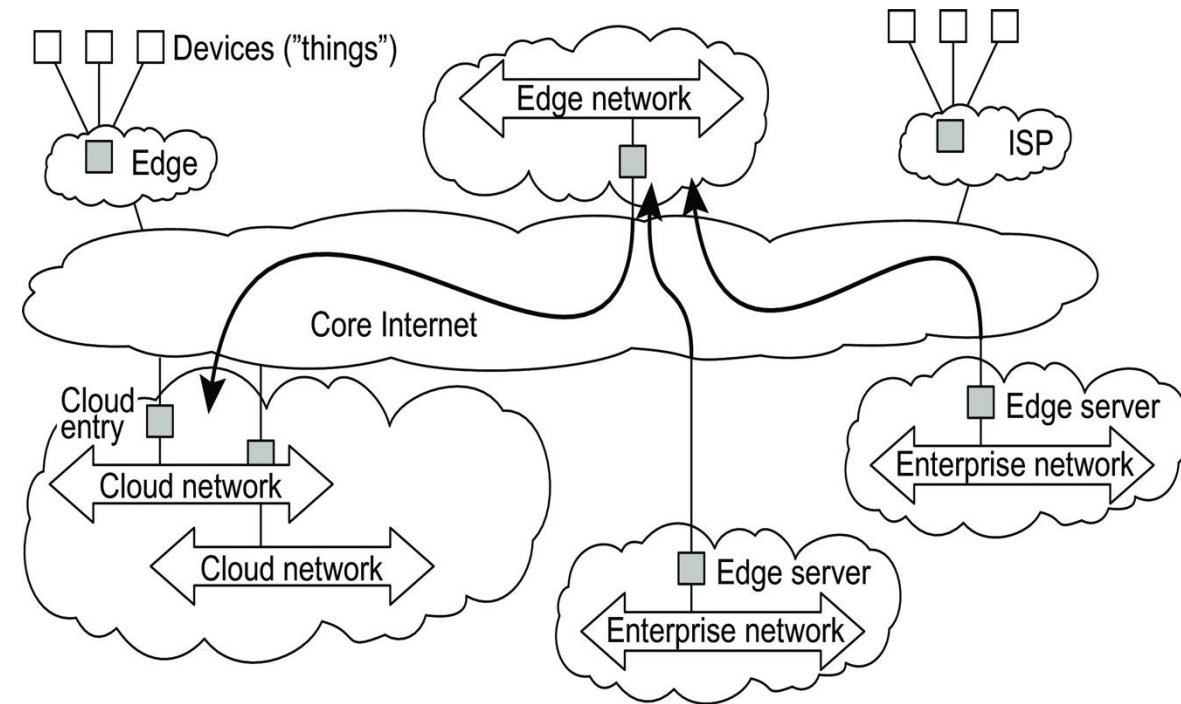
- Proxy files can become stale, while the main web server always has the most up-to-date version. Ensuring consistency across proxies is a key challenge. This can be handled either by proxies periodically requesting updates or by servers pushing updates when changes occur.
- Several factors affect update strategies:
 - **Data sensitivity** (e.g., medical or flight data) requires strict consistency, often with server-initiated updates.
 - **User demand** influences update frequency—low-demand files may be updated less often to reduce traffic.
 - **High-demand files** need more frequent consistency checks.
 - **Rapidly changing data** (e.g., stock prices) increases update traffic.

Designing efficient proxy systems depends on balancing data consistency, demand, and update frequency, while ensuring edge servers deliver better performance.

Edge Computing

- Web caches are effective when deployed closer to clients
 - At the “Edge” of the network
- Edge Computing: a paradigm where applications run on servers located at the edge of the network

Ex: mobile devices access edge servers hosted at cell towers
- Benefits
 - Significantly lower latency than “remote” cloud servers
 - Higher bandwidth
 - Can tolerate network or cloud failures
- Complements cloud computing
 - Cloud providers offer edge servers as well as cloud servers



Source: Steen & Tannenbaum textbook

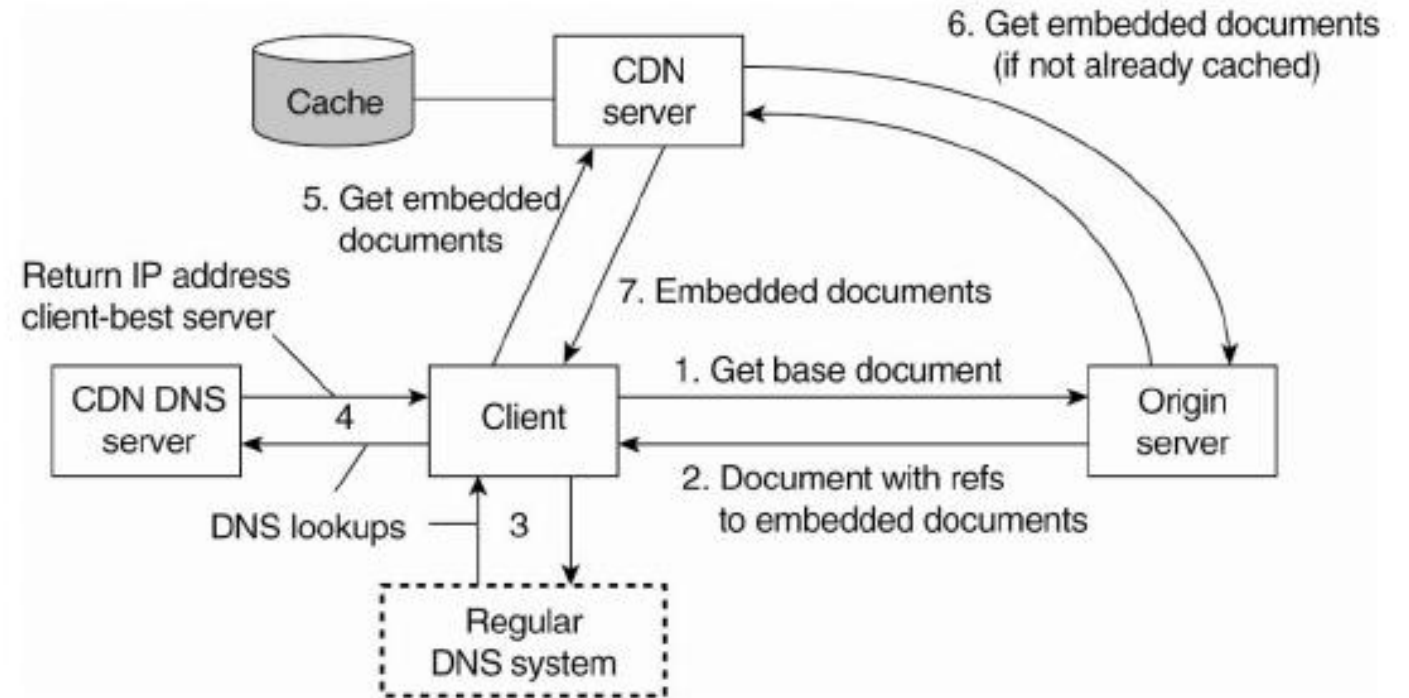
Content Distribution Networks (CDNs)

All modern large-scale computer systems use CDNs

- Content delivery networks
 - Network of edge caches deployed as commercial service
 - Cache web content (especially rich content: images, video)
 - Deliver from closest edge proxy server
- Global network of edge proxies to deliver web content
 - Edge clusters of varying sizes deployed in all parts of the world
 - [Ex:](#) Akamai CDN (120K servers in 1100 networks, 80 countries)

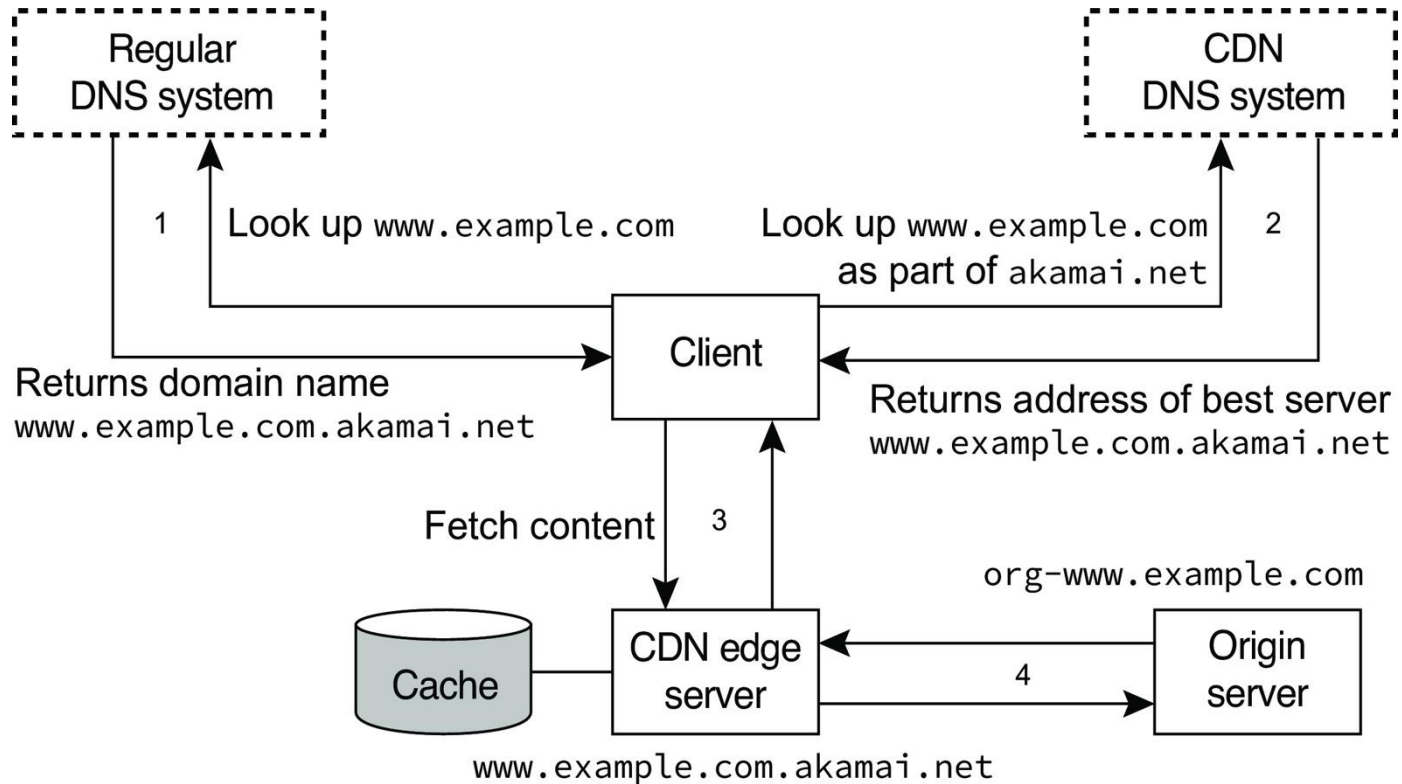
How do CDNs serve a request?

- CDNs try to serve a request using a cached file from a local server
- Regular DNS provides an IP address for the service
- CDN DNS (maintained by CDN) maps that IP address to the IP address of a nearby server



Source: Steen & Tannenbaum textbook

Working Principle of Akamai CDN



Source: Steen & Tannenbaum textbook

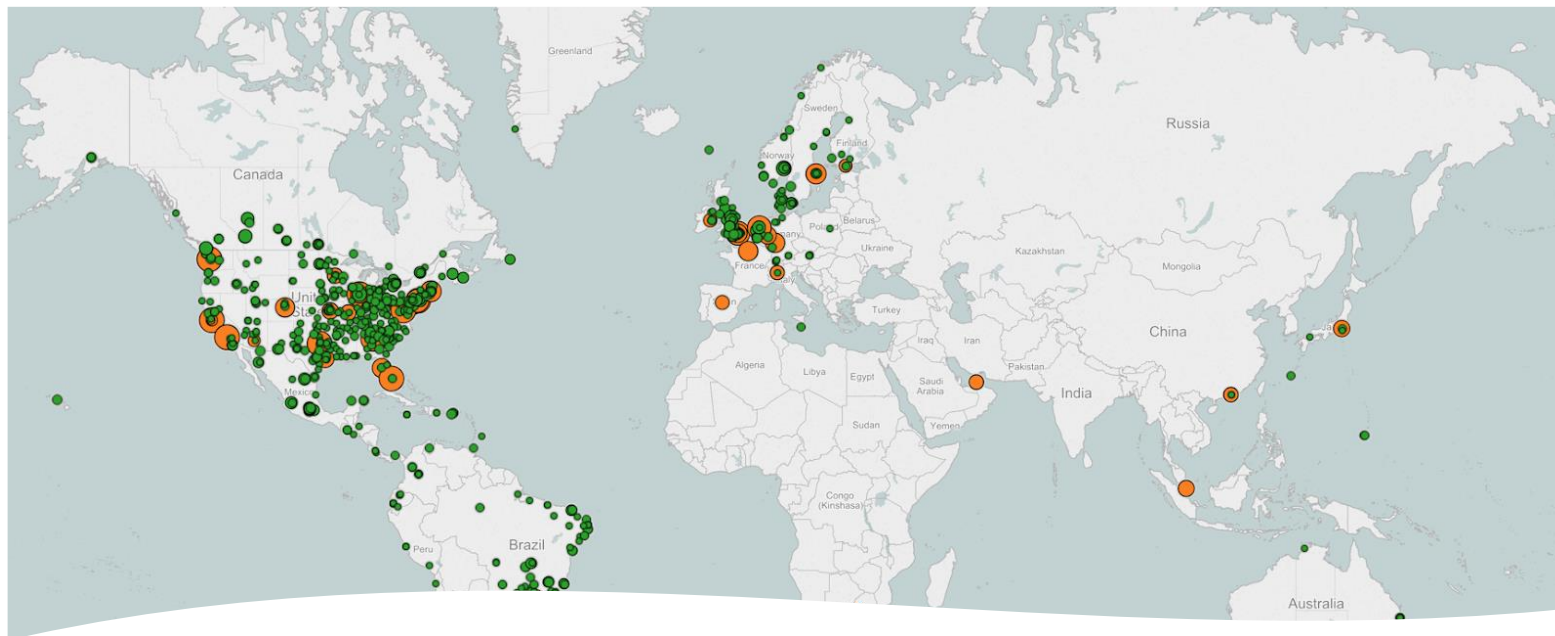
Extra Resources on CDNs (will not be assessed)

- Watch the following talk by Prof. Ramesh K. Sitaraman (UMass Amherst & Akamai) on the Networking Channel:
 - https://www.youtube.com/watch?v=1RQE8z_vQu0
- Microsoft 365 CDNs:
 - <https://learn.microsoft.com/en-us/microsoft-365/enterprise/content-delivery-networks?view=o365-worldwide>
- CDN Stands For Content Delivery Network: History and Evolution
 - <https://blog.blazingcdn.com/en-us/cdn-stands-for-content-delivery-network-history-and-evolution>

Netflix as a Distributed System

How Netflix streams movies to their users?





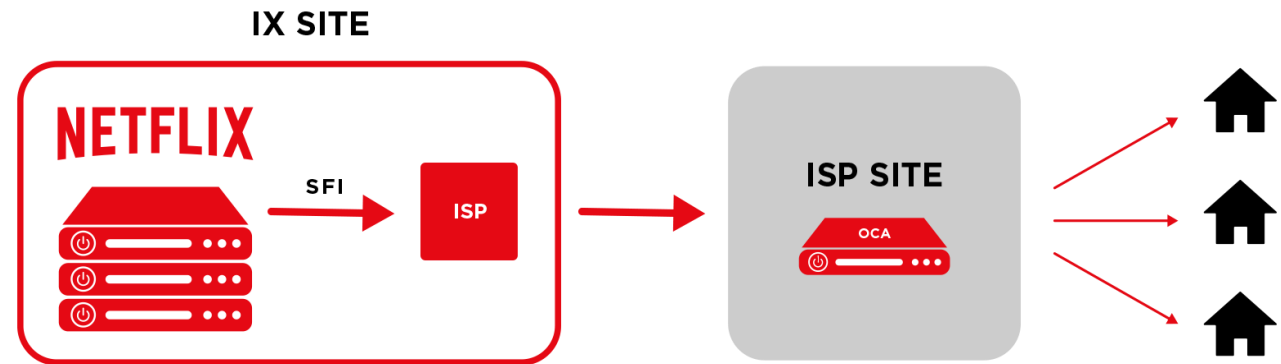
■ ISP Locations ■ Internet Exchange Point (circles are sized by volume)

Netflix Works With ISPs Around the Globe

- Netflix Open Connect (their CDN) delivers 100% of their video traffic, in 2020 – collective total of 6 billion hours per month
- Deliver via direct connections between Open Connect and the residential Internet Service Providers (ISPs)
- Give qualifying ISPs the same Open Connect Appliances (OCAs) - Close to 1,000 separate locations around the world, e.g., New York, Paris, London, Hong Kong, and Tokyo

NETFLIX Open Connect

- Eligible ISPs are provided with OCAs (local NETFLIX servers)
- OCAs and ISPs, which are part of an internet exchange point (IX), exchange files using SFI protocol during nighttime
- Customers access files from the OCA of the local ISP



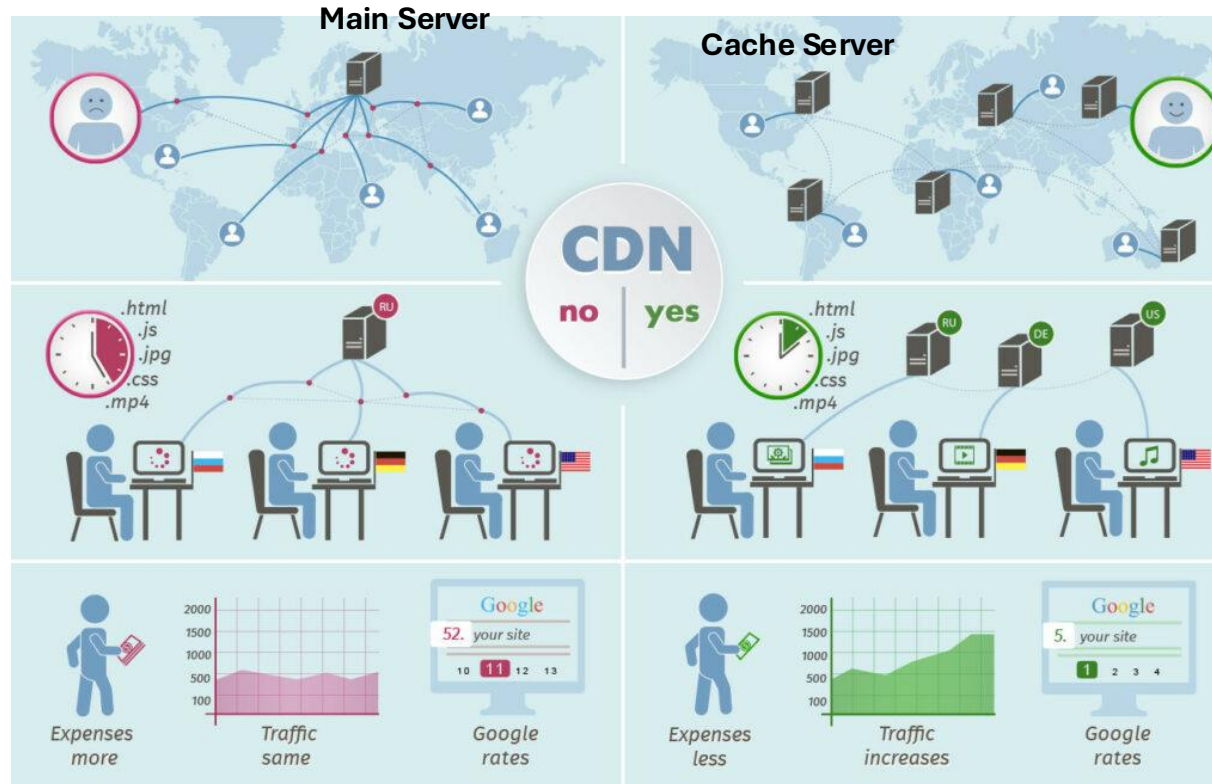
Source: Netflix Website

- <https://openconnect.netflix.com/Open-Connect-Overview.pdf>



CDN – Content Distribution Network

- Netflix's CDN is no different to other CDNs — except their CDN is dedicated to Netflix content.



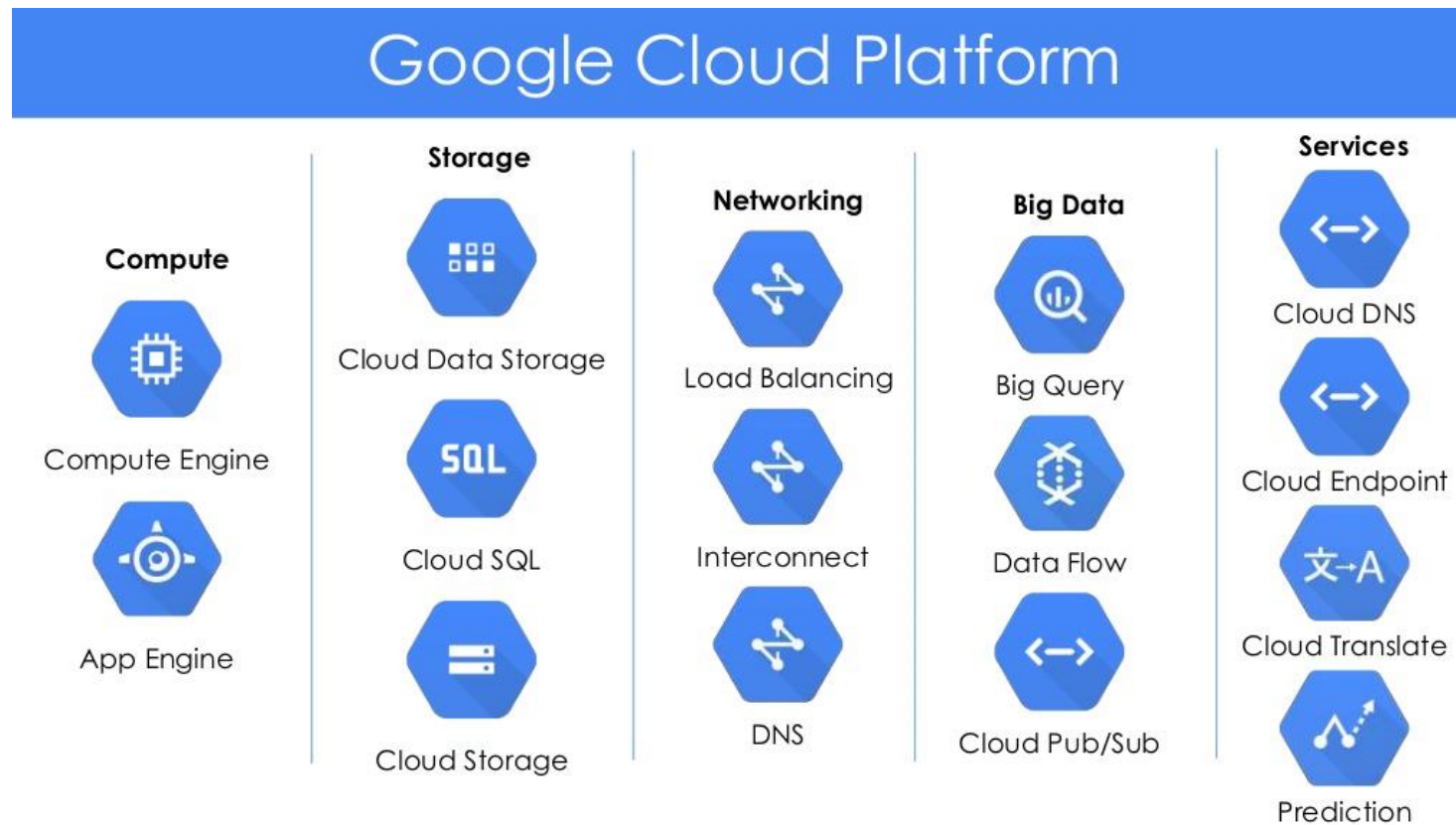
Amazon Web Services

- The Amazon Web Services (AWS) are a collection of remote computing services that together make up a cloud computing platform (<http://aws.amazon.com/>)
- Mainly infrastructure services:
 - Amazon Elastic Compute Cloud (EC2) – web services API for provisioning, managing, and deprovisioning virtual servers inside amazon cloud
 - Amazon Simple Storage Service (S3) – Instance or persists across time storage
 - Amazon Simple Queue Service (SQS) – transmit any volume of data without loss; for decoupling cloud components
 - Amazon CloudFront – Cloud-based content distributing network (content moves with the application)
 - Amazon SimpleDB – Relational database, Dropbox uses this



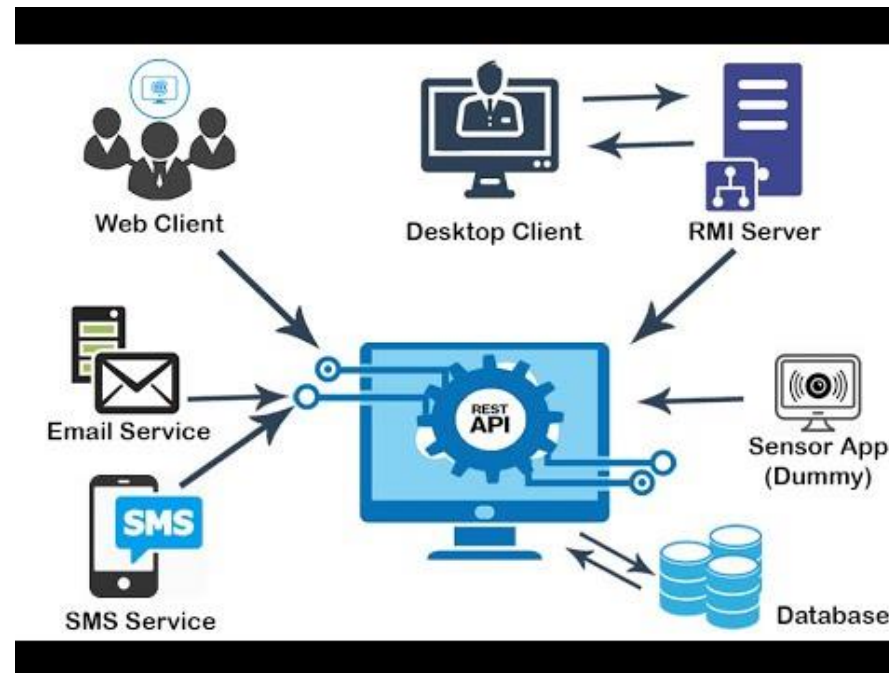
Google Web Services

- Google Developers - Web APIs make it possible to search through Google's extensive catalogue of Web content
(<http://developers.google.com>) (<https://cloud.google.com/?hl=en>)



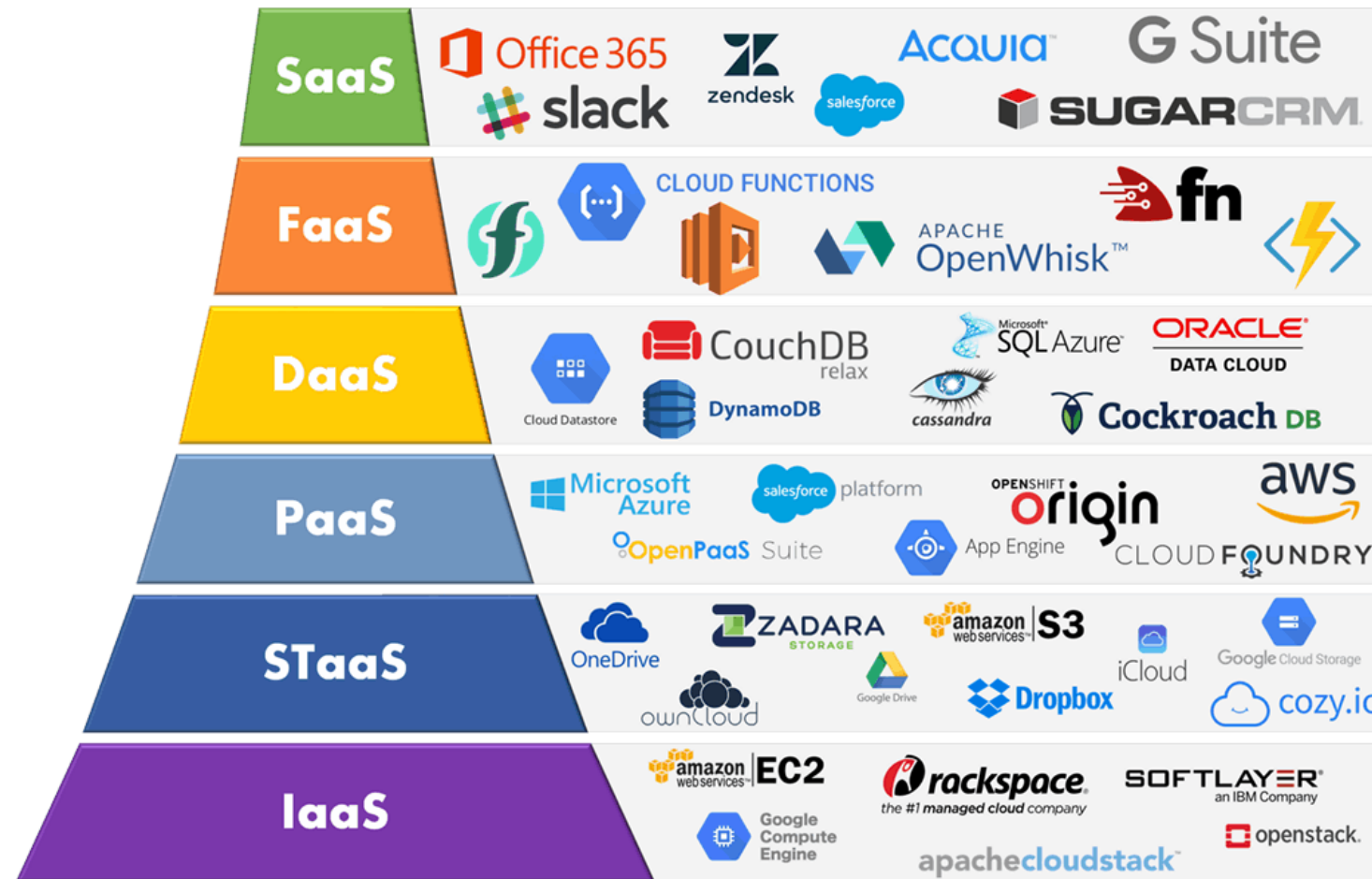
Integrating Web Services with RMI

- Web Services: One of the middleware technologies
- Based on HTTP communications yet offering RMI facilities
- Hence, it is suitable technology to form components of distributed systems, which are either internet-based or involving some internet connections.



Web Services from Different Vendors

- <https://kinsta.com/blog/google-cloud-vs-aws/>



Reference

- **Chapter 4 (for MOM) and Chapters 2,3,7 (for Web Services) of**
Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.
- **Chapters 8 and 9 of**
Distributed systems : concepts and design / George Coulouris ... [et al.], Addison-Wesley (Pearson), c2012.



COMP2211

Distributed Systems

Topic 4: Replication Architectures

Thiru Vasantam

This Topic

Aim

What is replication and what are its benefits?

Types of Architectures: Passive and Active
Replication Systems

Specific Issues and Solutions

Why Replication?

- **Replication:** We replicate data or functions over multiple components or servers.
- **Why replicate?**
 - Consider a distributed system in which we make a copy of a specific part of a system (its code and data).
 - **Increase reliability:** if one copy fails to provide service, then the other copy can provide service until the failed copy is restored
 - **Improve performance:** spread requests between different replicated parts to keep load balanced, or to ensure quick responses by taking proximity into account. You can increase the size of the distributed system (in terms of the number of servers or the geographic area).
 - **Examples:** All modern computer systems, such as Netflix and Google, use replication. We use web caching, proxy servers, and parallel processing in supercomputers.

Replication Issues

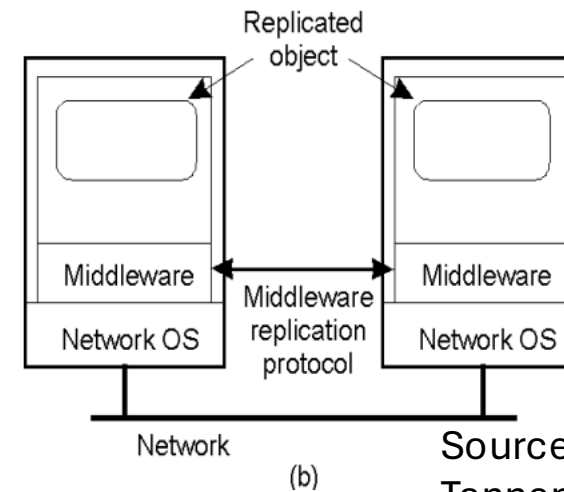
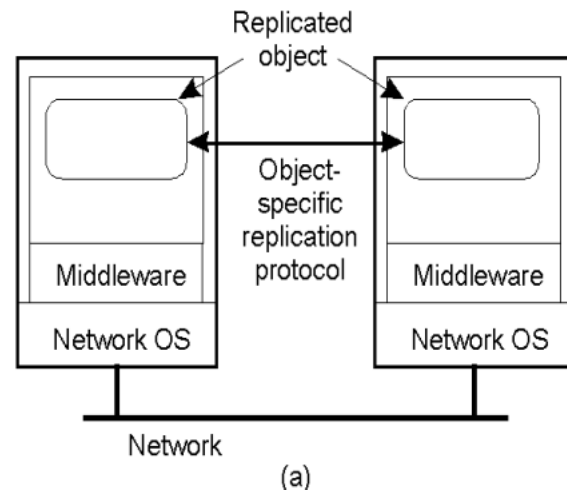
- When to Replicate?
- Where should we place replicas?
 - Consider the distance from users to servers or the rate of requests triggered by users
- How many replicas do we need?
 - Keep in mind the latency and other performance guarantees
 - Level of tolerance to the number of faults

These issues depend on the system performance and guarantees you want to provide.

Question: Can you think of any drawbacks of replication?

Ans: Achieving consistency is a major challenge

Two Approaches for Replication of Objects



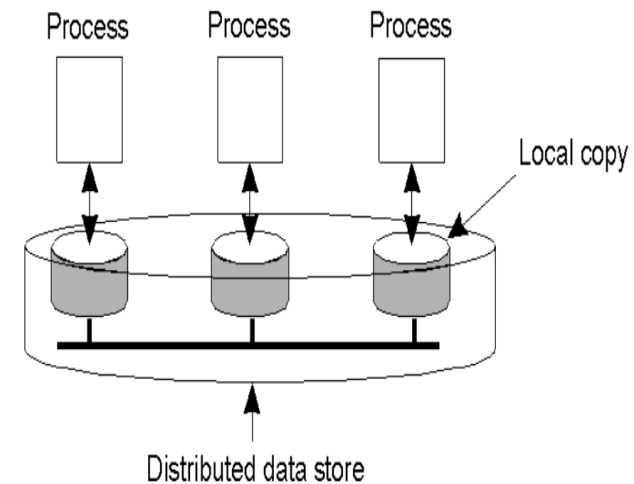
Source: Steen & Tannenbaum textbook

- **Method 1:** application is responsible for replication, need to update all copies
 - Application needs to handle consistency issues
- **Method 2:** system (middleware) handles replication
 - Consistency issues are handled by the middleware
 - Simplifies application development but makes object-specific solutions harder

Challenges

Consistency problem

- If we make changes to one copy, those changes should be made to all copies as soon as possible: replicas need to be identical, i.e., kept **consistent**.

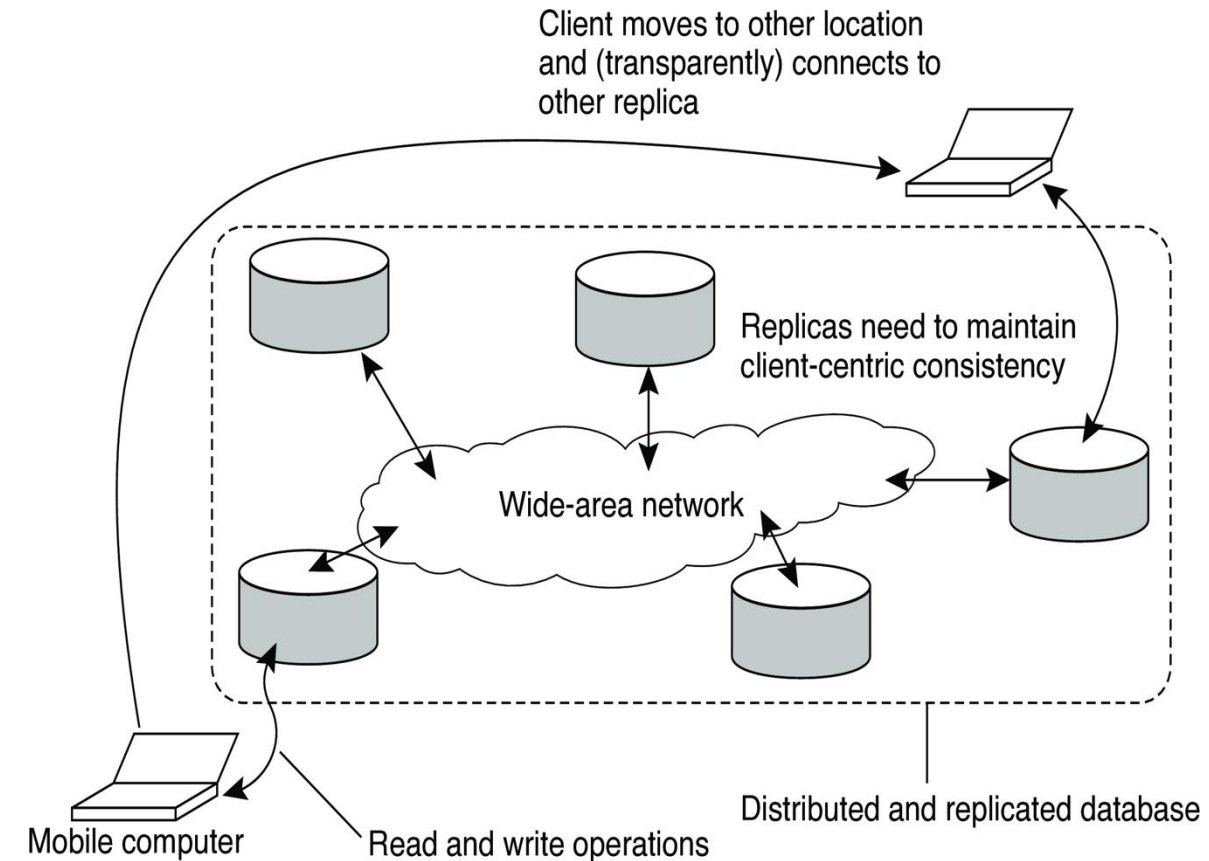


Source: Steen & Tannenbaum
textbook

Consistency for Mobile/Distributed Users

Example:

Consider a distributed database to which you have access through your notebook. Your notebook acts as a front end to the database.



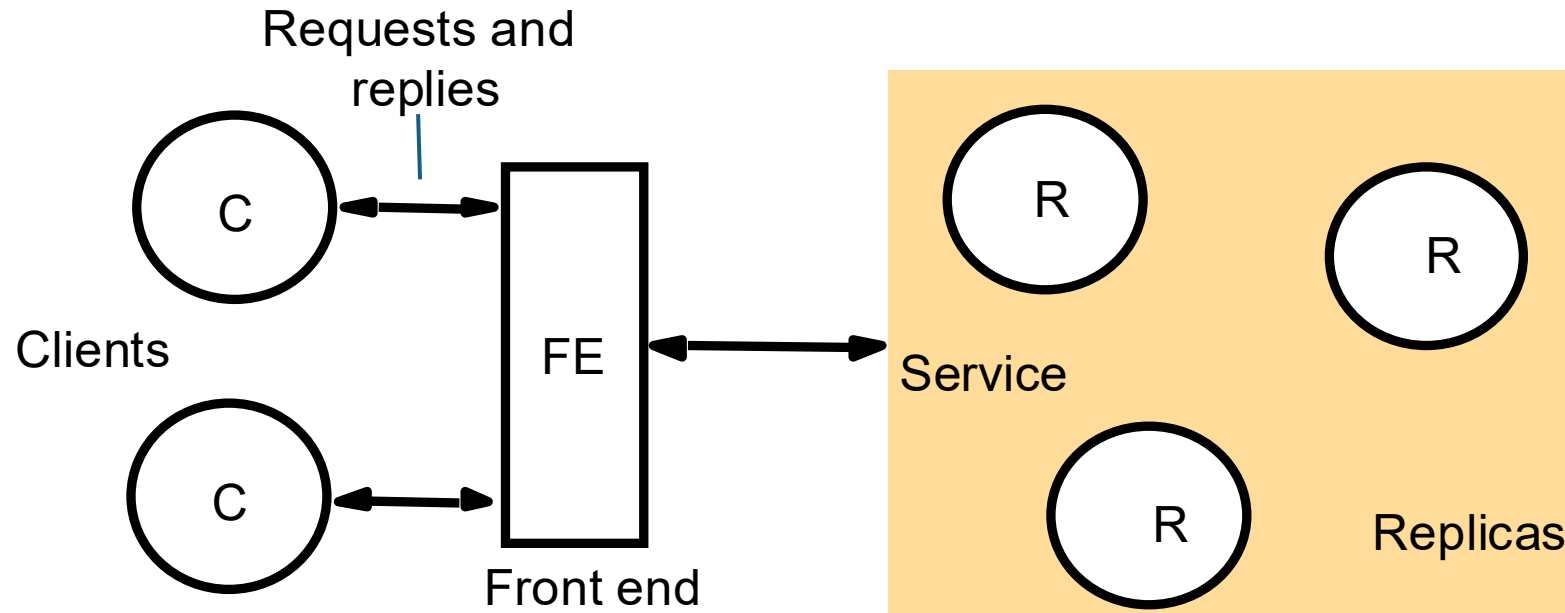
Source: Steen & Tannenbaum textbook

Data updates in Google Calendar and Dropbox experience some delays. Updates travel at most at the speed of light.

Types and Requirements

- Types:
 - Computation (function / service) replication: multiple instances of the same functional process are executed **(Behaviour)**
 - May run on different hardware, language, and implemented by different algorithms
 - Data replication: same piece of information is being stored in multiple devices **(Data integrity)**
 - Can be made available on-demand, i.e., data inconsistency, out of date, or missing data may be acceptable when not in use
- Depending on the type of faults/errors that we need to address:
 - Passive replication
 - Active replication

Base Model with Middleware– Replication

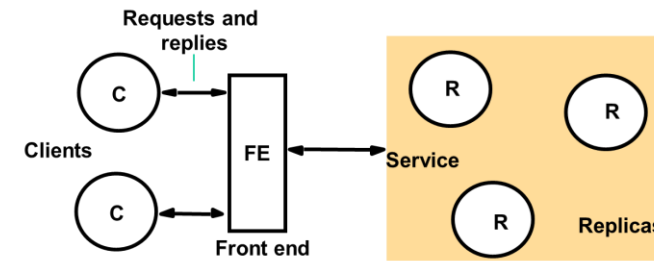


- Replicas
 - Maintain copies of the same data or functions; they can be implemented by different technologies
 - Replicas are not necessarily consistent all the time (some may have received updates, not yet conveyed to the others)
 - Example of inconsistency: Google Calendar updates in different devices

System Components

- Replicas (Rs) – **Back-end servers**
 - Maintain replicas (data / functions) on servers
 - Process requests or store results (may propagate to other servers)
 - Dynamic / static: set of Rs is fixed or variable (scalability issue)
- Clients (C) requests
 - those without updates are called **read-only** requests, the others are called **update or write** requests (they may include reads)
 - Read-only: handle by one replica
 - Update: may involve data propagation / synchronization, and concurrency control
- Front end (FE) – **Front-end server**
 - Make replication transparent
 - Monitor and maintain replica availability
 - Distribute client requests and collate responses

System Workflow



1. Incoming request

- Receive by the FE, and the FE will forward the request to R(s)

2. Coordination

- R(s) accepts a request
- Decide the ordering of a request relative to other requests

3. Execution

- R(s) processes the request

4. Agreement

- R(s) reach consensus on the effect of the requests

5. Response

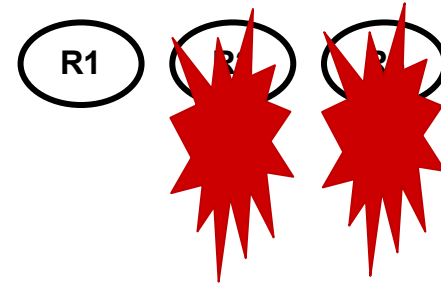
- One or more Rs reply to the FE
- FE may process the response before returning it to the client

Faults in Distributed Systems

Inconsistent results are obtained from the servers because of:

- **Omission failures** (e.g., crash failures, failing to receive a request, or failing to send a response) and;
- **Commission failures** (e.g., processing a request incorrectly, corrupting local state, and/or sending an incorrect or inconsistent response to a request) due to hardware malfunction, software bugs, or malicious attacks.

Fault-Tolerant Services

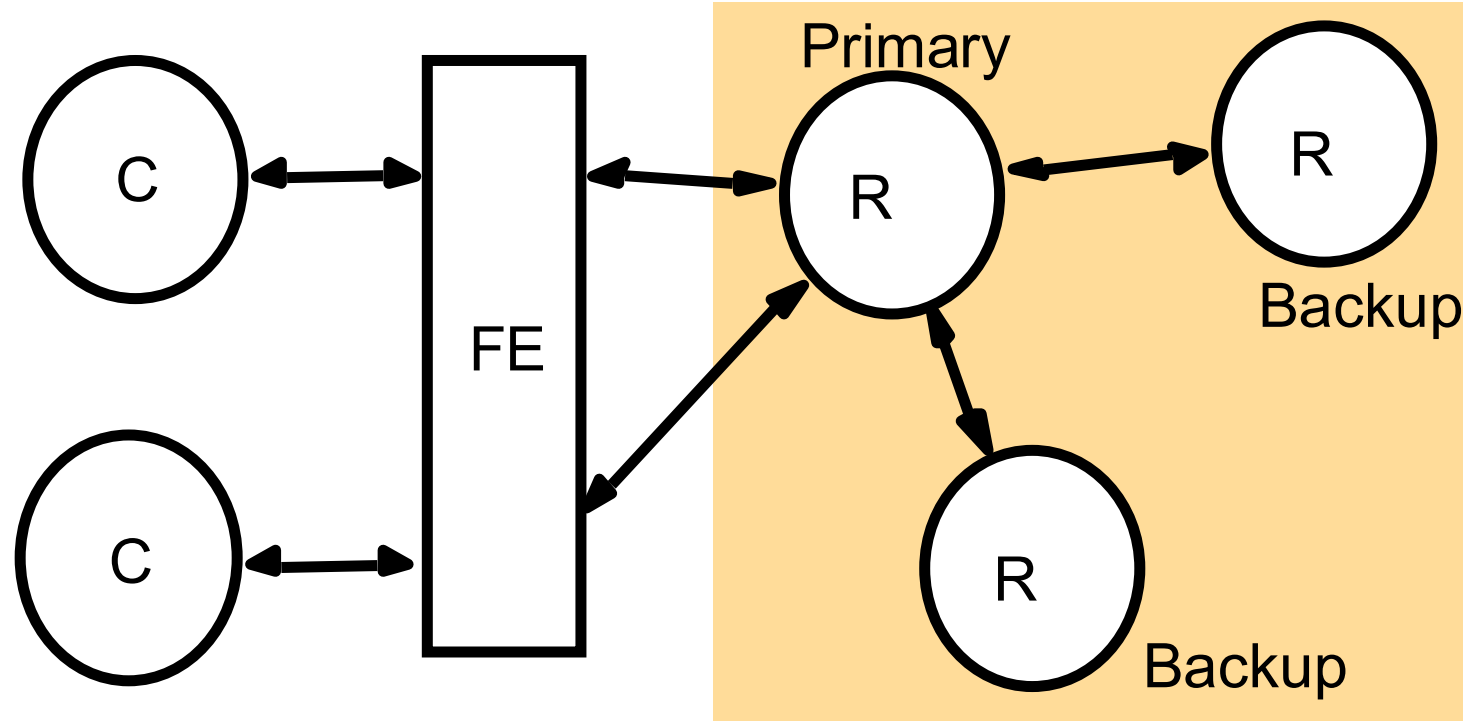


- Provide a correct service despite there being process/server failures, maybe up to ‘f’ process failures
- Each replica is assumed to behave according to the specification of the distributed system, when they have not crashed
 - e.g., a specification of bank accounts: ensure funds transferred between bank accounts can never disappear, and that only deposits and withdrawals affect the balance of an account.
- A service based on replication is **correct** if:
 - **Failure transparency:** it keeps responding despite failures
 - **Location / Relocation transparency:** if clients cannot tell the difference between the service they obtain from an implementation with replicated data and the one provided by a single correct replica manager. For example, the service provided by different processes should result in the same delays.

Passive Replication



Passive (primary-backup) model for fault tolerance



- There is at any time a single primary R and one or more secondary (backup, slave) Rs
- FEs communicate with the primary which executes the operation and sends copies of the updated **data** to backups
- if the primary fails, one of the backups is promoted to act as the primary

How many replicas should be there to have a fault-tolerant service to at most K server/replica crashes?

Ans: We need K+1 replicas, even if K servers crash, the remaining active server can provide service.

Workflow of Passive replication

- 1. Request:
 - A FE issues the request, containing a unique identifier, to the primary R
- 2. Coordination:
 - The primary processes each request atomically, in the order in which it receives it relative to other requests (*Message ordering*)
 - It checks the unique id; if it has already done the request, it re-sends the response. (*Handle Message Loss*)
- 3. Execution:
 - The primary executes the request and stores the response
- 4. Agreement:
 - If the request is an update, the primary sends the updated state, the response and the unique identifier to all the backups. The backups send an acknowledgement (*Result propagation*)
- 5. Response:
 - The primary responds to the FE, which hands the response back to the client

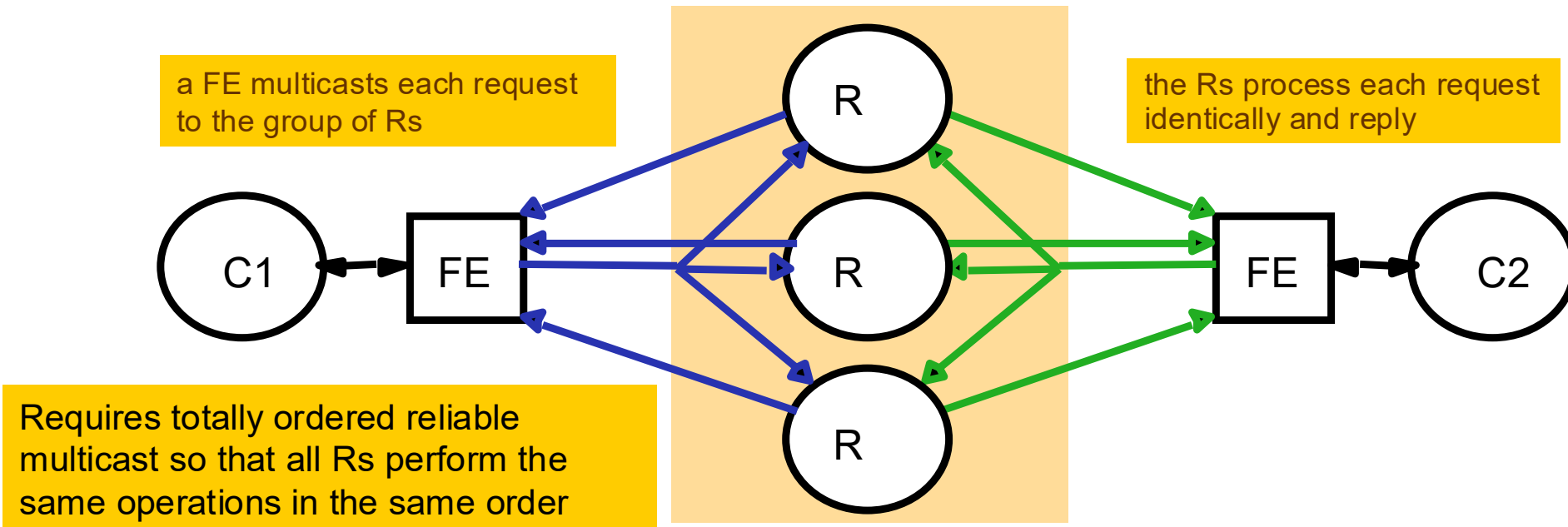
Discussion of Passive Replication

- Replica crashes
 - Can provide service or Survive up to f replica crash, when the system comprises $f + 1$ replicas. Here, when a replica/server crashes it cannot provide any service (data/results).
- Front-end functionality
 - Requires little functionality: only need to lookup a new primary replica when the current one is not available.
- System overhead
 - Relatively large due to data propagation from primary to all other replicas.

Active Replication

Active replication

- The Rs are *state machines* all playing the same role and organised as a group.
 - All start in the same state and perform the same task in the same order so that their state remains identical (**synchronization**)
- If an R crashes it has no effect on performance of the service because the others continue as normal



Workflow of Active replication

- 1. Request
 - FE attaches a unique *id* and uses *totally ordered reliable multicast* to send request to Rs. FE can at worst, crash. It does not issue requests in parallel (*Performance and reliability issues*)
- 2. Coordination (*Message ordering*)
 - The multicast delivers requests to all the Rs in the same (total) order.
- 3. Execution
 - Every R executes the request. They are state machines and receive requests in the same order, so the effects are identical. The *id* is put in the response
- 4. Agreement
 - No agreement is required because all Rs execute the same operations in the same order, due to the properties of the totally ordered multicast.
- 5. Response
 - FEs collect responses from Rs. FE may just use one or more responses. If it is only trying to tolerate crash failures, it returns the first response to the client. (*Improve scalability*)

Byzantine fault

- A **Byzantine fault** is an arbitrary fault that occurs during the execution of an algorithm when multiple servers are involved in producing results for the same client request. The server is running, but it produces the wrong result. These errors are difficult to handle.
- **Byzantine failure-tolerant algorithms**: characterised by their resilience f , the number of faulty processes with which an algorithm can cope. The majority decision of R_s is used for response.

Question: How many R_s do you need for fault-tolerance of up to f Byzantine failures?

Ans: We need $2f+1$ servers as the final response is decided based on the majority decision (f correct responses and $f+1$ wrong responses)

Discussion of Active Replication

- Assumption
 - A solution to totally ordered and reliable multicast is available
- Fail situation
 - Can mask up to f **Byzantine failures**, if the system incorporates at least $2f + 1$ replicas
 - Front end waits until it has collected $f + 1$ identical responses and passes that response back to the client, and discards other responses to the same request
- Read-only request
 - Front-end may send read-only requests only to individual replica
 - Lose fault tolerance, but remain sequentially consistent
 - Can easily mask replica failure in this case, by submitting the read-only request to another replica

Passive Replication V/S Active Replication

- **Passive Replication:**

- It provides strong consistency as all the requests for updates are processed by the primary server, which then sends updates to back-up servers
- Fault-tolerance is provided as back-up servers can provide service when the primary fails
- Back-up servers are not utilised, hence, leads to a wastage of resources. They are used only when the primary fails.
- Simple to implement, and it is used in applications which require strong consistency and fault tolerance.

- **Active Replication:**

- Complex to implement as it requires a totally-ordered reliable multi-cast transmission of data
- Data could be inconsistent as all the servers process update requests, sometimes they can have errors due to failure of totally-ordered multi-cast
- Used when high availability of servers is required, can provide service even if some servers fail, as all the servers try to process requests when assigned.

Implementation: Server 1

```
import Pyro4

@Pyro4.expose
class ReplicationServer1:
    def do_something(self, data):
        # Perform some operation on the data
        result = data + 1
        return result

daemon = Pyro4.Daemon()
ns = Pyro4.locateNS()
uri = daemon.register(ReplicationServer1)
ns.register("replication1", uri)
print("Replication Server 1 ready.")
daemon.requestLoop()
```

Implementation: Server2

```
import Pyro4
```

```
@Pyro4.expose
```

```
class ReplicationServer2:
```

```
    def do_something(self, data):
```

```
        # Perform some operation on the data
```

```
        result = data + 2
```

```
        return result
```

```
daemon = Pyro4.Daemon()
```

```
ns = Pyro4.locateNS()
```

```
uri = daemon.register(ReplicationServer2)
```

```
ns.register("replication2", uri)
```

```
print("Replication Server 2 ready.")
```

```
daemon.requestLoop()
```

Implementation: Front-End

```
import Pyro4
```

```
class FrontendServer:
```

```
    def __init__(self):
```

```
        self.replication1 = Pyro4.Proxy("PYRONAME:replication1")
```

```
        self.replication2 = Pyro4.Proxy("PYRONAME:replication2")
```

```
    def do_something(self, data):
```

```
        result1 = self.replication1.do_something(data)
```

```
        result2 = self.replication2.do_something(data)
```

```
        return result1, result2
```

```
daemon = Pyro4.Daemon()
```

```
ns = Pyro4.locateNS()
```

```
frontend = FrontendServer()
```

```
uri = daemon.register(frontend)
```

```
ns.register("frontend", uri)
```

```
print("Frontend Server ready.")
```

```
daemon.requestLoop()
```

Implementation: Client

```
import Pyro4
```

```
frontend = Pyro4.Proxy("PYRONAME:frontend")
```

```
result1, result2 = frontend.do_something(10)
```

```
print("Result from Replication Server 1:", result1)
```

```
print("Result from Replication Server 2:", result2)
```

Reference

- **Chapter 7 of**
Distributed Systems, 4th Edition,
Maarten van Steen and Andrew
Tannenbaum.
- **Chapter 18 of**
Distributed systems : concepts and
design / George Coulouris ... [et al.],
Addison-Wesley (Pearson), c2012.



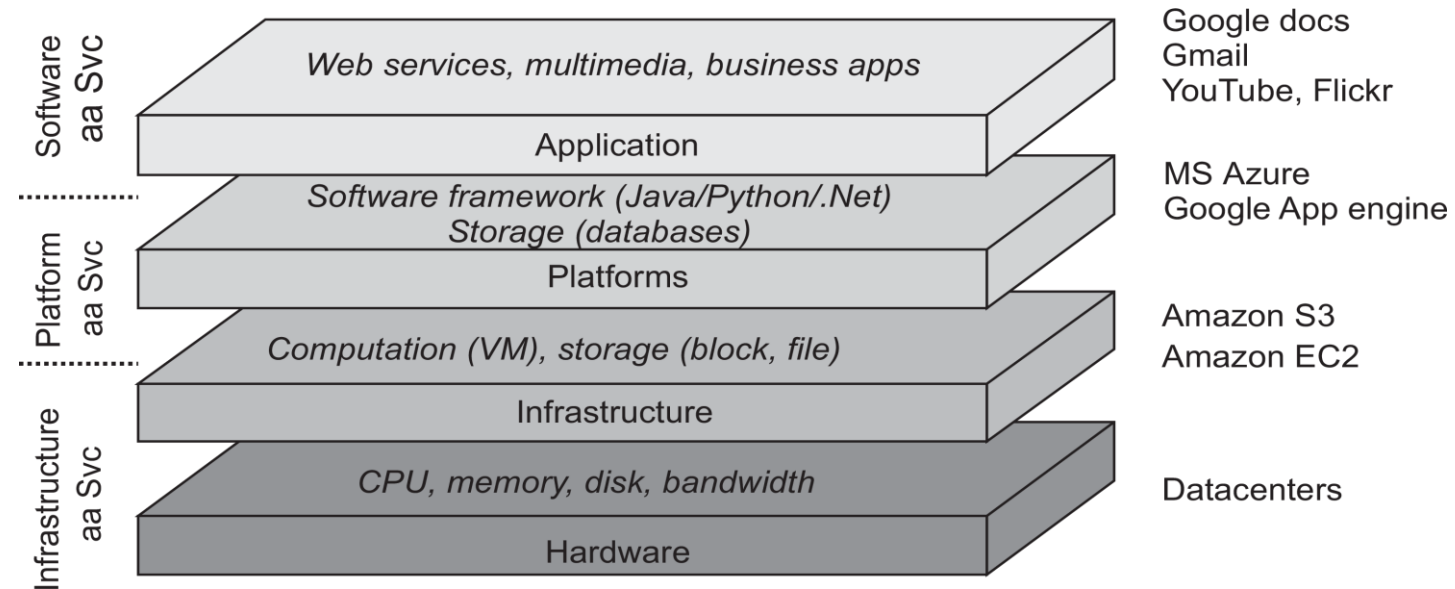
COMP2211

Distributed Systems

Topic 5: Cloud Computing and Load Balancing

Thiru Vasantam

Cloud Computing



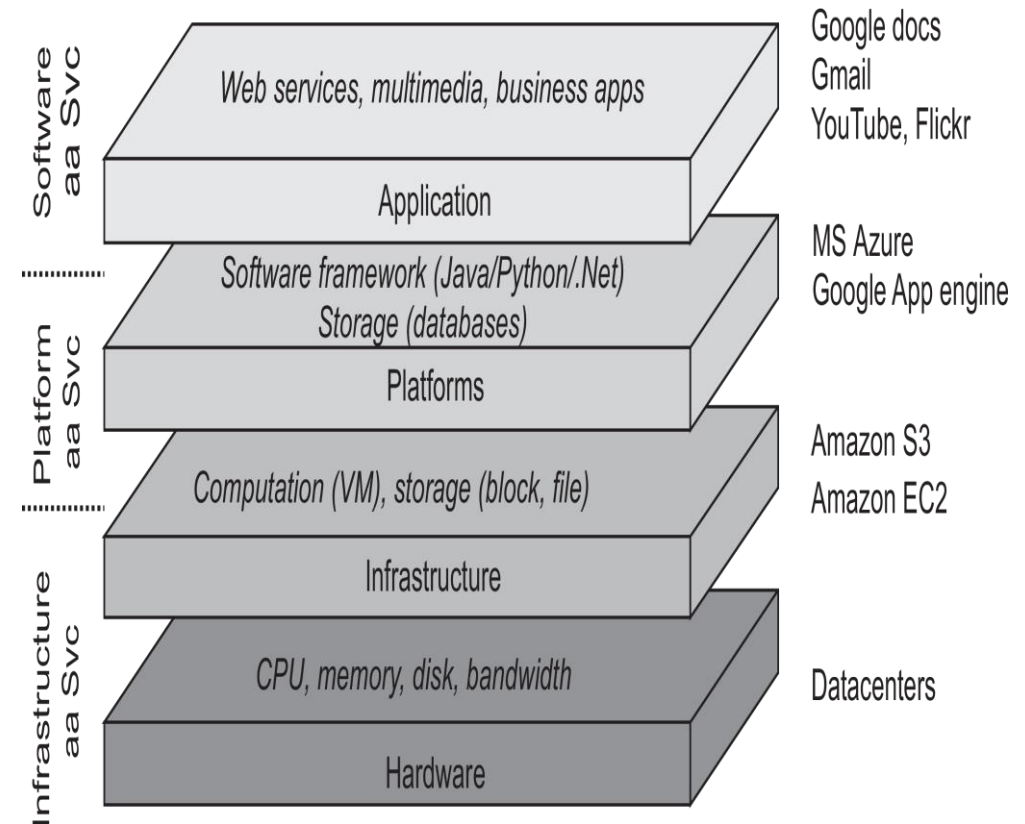
- **Cloud computing:** use of remote servers to run distributed applications
- Cloud computing platform: Data center where remote resources can be leased by any user or company
 - No need to create and deploy own data center and IT infrastructure

Source: Steen and Tannenbaum textbook

Cloud Computing

Make a distinction between four layers

- **Hardware:** Processors, routers, power and cooling systems. Customers normally never get to see these.
- **Infrastructure:** Deploys virtualization techniques. Evolves around allocating and managing virtual storage devices and virtual servers.
- **Platform:** Provides higher-level abstractions for storage and such. **Example:** Amazon S3 storage system offers an API for (locally created) files to be organised and stored in so-called **buckets**.
- **Application:** Actual applications, such as office suites (text processors, spreadsheet applications, presentation applications).



Reference

- Chapter 3 of *Distributed Systems, 4th Edition, Maarten van Steen and Andrew Tannenbaum.*
- Niranjan G. Shivaratri , Phillip Krueger , Mukesh Singhal, Load Distributing for Locally Distributed Systems, Computer, v.25 n.12, p.33-44, December 1992
- V. Cardellini and E. Casalicchio, “The State of the Art in Locally Distributed Web-Server Systems,” *ACM Computing Surveys*, **34**(2):170-177, June 2002.

